

GROUPING SETS, ROLLUP, and CUBE in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: Multi-Level Aggregation](#)
 3. [GROUPING SETS](#)
 4. [ROLLUP](#)
 5. [CUBE](#)
 6. [The GROUPING\(\) Function](#)
 7. [Combining GROUPING SETS](#)
 8. [ASCII Visual Diagrams](#)
 9. [Practical Report Examples](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions & Answers](#)
 14. [Hands-on Exercises](#)
 15. [Advanced Notes](#)
 16. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use GROUPING SETS for flexible multi-level aggregation
 - Use ROLLUP to generate subtotals and grand totals
 - Use CUBE to generate all possible grouping combinations
 - Distinguish subtotal NULLs from data NULLs using GROUPING()
 - Build executive dashboards and cross-tab reports in a single query
-

Theory: Multi-Level Aggregation

Traditional GROUP BY produces one result per unique combination of grouping columns. For reports that need subtotals, grand totals, or cross-tabs, you would historically use multiple queries joined with UNION ALL. GROUPING SETS, ROLLUP, and CUBE replace this with a single query.

The Problem UNION ALL Solves (and Then GROUPING SETS Replaces)

```
-- OLD WAY: Multiple GROUP BY queries + UNION ALL
SELECT region, product, SUM(amount) FROM sales GROUP BY region, product
UNION ALL
SELECT region, NULL, SUM(amount) FROM sales GROUP BY region
UNION ALL
SELECT NULL, product, SUM(amount) FROM sales GROUP BY product
UNION ALL
SELECT NULL, NULL, SUM(amount) FROM sales;

-- NEW WAY: One query with GROUPING SETS
```

```
SELECT region, product, SUM(amount)
FROM sales
GROUP BY GROUPING SETS ((region, product), (region), (product), ());
```

GROUPING SETS

GROUPING SETS allows you to specify exactly which groupings you want. Each set in parentheses defines one grouping level.

Syntax

```
SELECT col1, col2, AGG(col3)
FROM table
GROUP BY GROUPING SETS (
    (col1, col2),    -- group by both
    (col1),          -- group by col1 only
    (col2),          -- group by col2 only
    ()               -- grand total (empty set)
);
```

Examples

```
-- Example 1: Region + Product detail, region subtotals, grand total
SELECT
    COALESCE(region, 'ALL REGIONS') AS region,
    COALESCE(product, 'ALL PRODUCTS') AS product,
    COUNT(*) AS sales_count,
    SUM(amount) AS revenue
FROM sales
WHERE region IS NOT NULL
GROUP BY GROUPING SETS (
    (region, product),
    (region),
    ()
)
ORDER BY
    GROUPING(region),
    region,
    GROUPING(product),
    product;

-- Example 2: Three-level GROUPING SETS
SELECT
    rep_name,
    region,
    product,
    SUM(amount) AS revenue
FROM sales
WHERE region IS NOT NULL
```

```

GROUP BY GROUPING SETS (
    (rep_name, region, product), -- detailed
    (rep_name, region),         -- rep+region subtotal
    (rep_name),                 -- rep total
    ()                          -- grand total
)
ORDER BY
    GROUPING(rep_name),
    rep_name,
    GROUPING(region),
    region,
    GROUPING(product),
    product;

-- Example 3: Cross-type grouping for a dashboard
SELECT
    DATE_TRUNC('month', sale_date) AS month,
    region,
    SUM(amount) AS revenue,
    COUNT(*) AS transactions
FROM sales
WHERE region IS NOT NULL
GROUP BY GROUPING SETS (
    (DATE_TRUNC('month', sale_date), region),
    (DATE_TRUNC('month', sale_date)),
    (region),
    ()
)
ORDER BY GROUPING(month), month, GROUPING(region), region;

```

ROLLUP

ROLLUP generates subtotals along a hierarchy from left to right, culminating in a grand total. With N columns, ROLLUP generates N+1 groupings.

ROLLUP Expansion

```

ROLLUP(A, B, C) expands to GROUPING SETS:
(A, B, C)    ← detail
(A, B)       ← B subtotal removed
(A)          ← B and C subtotals removed
()           ← grand total

```

N columns → N+1 grouping levels

ASCII Diagram

```
ROLLUP(region, product):
```

```
Level 0 (detail):           Level 1 (region subtotal):  Level 2 (grand total):
```

region	product	rev	region	NULL	total	NULL	NULL
East	Wgt A	3000	East		5350		
East	Wgt B	950	North		4100		12250
East	Wgt C	1400	South		3900		
North	Wgt A	3300	West		2500		
...	...	...					

Examples

```
-- Example 4: ROLLUP – region > product hierarchy
SELECT
    COALESCE(region, 'GRAND TOTAL') AS region,
    COALESCE(product, 'SUBTOTAL') AS product,
    SUM(amount) AS revenue,
    COUNT(*) AS sales_count
FROM sales
WHERE region IS NOT NULL
GROUP BY ROLLUP(region, product)
ORDER BY
    GROUPING(region),
    region,
    GROUPING(product),
    product;

-- Example 5: ROLLUP for time hierarchy (year > month > day)
SELECT
    EXTRACT(YEAR FROM sale_date) AS yr,
    EXTRACT(MONTH FROM sale_date) AS mo,
    EXTRACT(DAY FROM sale_date) AS dy,
    SUM(amount) AS revenue
FROM sales
GROUP BY ROLLUP(
    EXTRACT(YEAR FROM sale_date),
    EXTRACT(MONTH FROM sale_date),
    EXTRACT(DAY FROM sale_date)
)
ORDER BY
    GROUPING(EXTRACT(YEAR FROM sale_date)),
    yr,
    GROUPING(EXTRACT(MONTH FROM sale_date)),
    mo,
    GROUPING(EXTRACT(DAY FROM sale_date)),
    dy;

-- Example 6: ROLLUP for a sales report with GROUPING() labels
SELECT
    CASE
        WHEN GROUPING(region) = 1 AND GROUPING(rep_name) = 1 THEN 'GRAND TOTAL'
```

```

        WHEN GROUPING(rep_name) = 1 THEN region || '
SUBTOTAL '
        ELSE region
    END AS region_label,
    CASE
        WHEN GROUPING(rep_name) = 1 THEN '-'
        ELSE rep_name
    END AS rep_label,
    SUM(amount) AS revenue
FROM sales
WHERE region IS NOT NULL
GROUP BY ROLLUP(region, rep_name)
ORDER BY
    GROUPING(region), region,
    GROUPING(rep_name), rep_name;

```

CUBE

CUBE generates ALL possible grouping combinations of the listed columns. With N columns, CUBE generates 2^N groupings.

CUBE Expansion

CUBE(A, B, C) expands to GROUPING SETS:

(A, B, C)	← all three
(A, B)	← C removed
(A, C)	← B removed
(B, C)	← A removed
(A)	← B and C removed
(B)	← A and C removed
(C)	← A and B removed
()	← grand total

N=3 columns → $2^3 = 8$ groupings

N=4 columns → $2^4 = 16$ groupings

Examples

```

-- Example 7: CUBE – all combinations of region, product, rep
SELECT
    region,
    product,
    rep_name,
    SUM(amount) AS revenue,
    COUNT(*) AS sales,
    GROUPING(region) AS g_region,
    GROUPING(product) AS g_product,
    GROUPING(rep_name) AS g_rep
FROM sales
WHERE region IS NOT NULL

```

```

GROUP BY CUBE(region, product, rep_name)
ORDER BY g_region, region, g_product, product, g_rep, rep_name;

-- Example 8: CUBE for cross-tab analysis (2 dimensions)
SELECT
    COALESCE(region, 'TOTAL') AS region,
    COALESCE(product, 'TOTAL') AS product,
    SUM(amount) AS revenue
FROM sales
WHERE region IS NOT NULL
GROUP BY CUBE(region, product)
ORDER BY
    GROUPING(region), region,
    GROUPING(product), product;

```

The GROUPING() Function

GROUPING() distinguishes between a NULL that represents a subtotal aggregate and a NULL that is actual data.

```

GROUPING(column)
-- Returns 0 if the column is used in the current grouping level
-- Returns 1 if the column is NULL because it is a rollup/cube aggregate

```

Examples

```

-- Example 9: Use GROUPING() to label rows
SELECT
    region,
    product,
    SUM(amount) AS revenue,
    GROUPING(region) AS is_region_subtotal,
    GROUPING(product) AS is_product_subtotal,
    GROUPING(region) + GROUPING(product) AS subtotal_level
FROM sales
WHERE region IS NOT NULL
GROUP BY CUBE(region, product)
ORDER BY subtotal_level, region, product;

-- Example 10: GROUPING_ID() – a single integer encoding all GROUPING() bits
SELECT
    region,
    product,
    SUM(amount) AS revenue,
    GROUPING_ID(region, product) AS grp_id
    -- grp_id=0: detail, grp_id=1: region subtotal, grp_id=2: product subtotal,
    -- grp_id=3: grand total
FROM sales
WHERE region IS NOT NULL

```

```

GROUP BY CUBE(region, product)
ORDER BY grp_id, region, product;

-- Example 11: Handle data NULLs vs subtotal NULLs in same column
-- (sales table has some NULL regions – how to tell them from ROLLUP NULLs?)
SELECT
    CASE
        WHEN GROUPING(region) = 1 THEN '[GRAND TOTAL]'
        WHEN region IS NULL      THEN '[Unknown Region]'
        ELSE region
    END AS region_display,
    SUM(amount) AS revenue
FROM sales
GROUP BY ROLLUP(region)
ORDER BY GROUPING(region), region NULLS LAST;

```

Combining GROUPING SETS

Multiple GROUPING SETS clauses can be combined (though typically ROLLUP/CUBE is cleaner):

```

-- Example 12: Partial ROLLUP using GROUPING SETS
-- Rollup of (region, product) but independent grouping by rep
SELECT region, product, rep_name, SUM(amount)
FROM sales
WHERE region IS NOT NULL
GROUP BY
    GROUPING SETS((region, product), (region), ()),
    GROUPING SETS((rep_name), ())
-- This is a cross-product of the two GROUPING SETS
ORDER BY 1, 2, 3;

```

ASCII Visual Diagrams

ROLLUP vs CUBE vs GROUPING SETS

Dimensions: Region (R), Product (P)

ROLLUP(R, P):	CUBE(R, P):	GROUPING SETS((R,P),(R),()):
(R, P) ← detail	(R, P) detail	(R, P) detail
(R) ← subtotal	(R) R-only	(R) subtotal
() ← grand total	(P) P-only	() grand total
	() grand total	
3 groupings	4 groupings	3 groupings
(N+1 for N=2)	(2^N for N=2)	(custom)

GROUPING() Bits

CUBE(A, B):

Row type	GROUPING(A)	GROUPING(B)	GROUPING_ID(A,B)
Detail	0	0	0
A subtotal	0	1	1
B subtotal	1	0	2
Grand total	1	1	3

Practical Report Examples

```
-- Example 13: Executive Sales Dashboard
SELECT
  TO_CHAR(DATE_TRUNC('month', sale_date), 'YYYY-MM') AS period,
  region,
  rep_name,
  SUM(amount) AS revenue,
  COUNT(*) AS transactions,
  ROUND(AVG(amount), 2) AS avg_deal_size,
  CASE
    WHEN GROUPING(rep_name) = 1 AND GROUPING(region) = 1 THEN 'GRAND TOTAL'
    WHEN GROUPING(rep_name) = 1 THEN 'REGION SUBTOTAL'
    ELSE 'DETAIL'
  END AS row_type
FROM sales
WHERE region IS NOT NULL
  AND sale_date >= '2024-01-01'
GROUP BY
  ROLLUP(DATE_TRUNC('month', sale_date), region, rep_name)
ORDER BY
  GROUPING(DATE_TRUNC('month', sale_date)),
  DATE_TRUNC('month', sale_date),
  GROUPING(region), region,
  GROUPING(rep_name), rep_name;
```

Common Mistakes

Mistake 1: Confusing subtotal NULLs with data NULLs

```
-- WRONG: treating all NULLs as missing data
SELECT COALESCE(region, 'Unknown') AS region, SUM(amount)
FROM sales
GROUP BY ROLLUP(region);
-- 'Unknown' could mean real NULL data OR a rollup aggregate!

-- CORRECT: use GROUPING()
SELECT
```



```
CASE WHEN GROUPING(region) = 1 THEN 'GRAND TOTAL'
      WHEN region IS NULL THEN 'Unknown Region'
      ELSE region
END AS region,
SUM(amount)
FROM sales
GROUP BY ROLLUP(region);
```

Mistake 2: Too many CUBE dimensions — combinatorial explosion

```
-- CUBE(A, B, C, D) = 2^4 = 16 groupings
-- CUBE(A, B, C, D, E) = 2^5 = 32 groupings
-- Avoid CUBE with more than 3-4 dimensions in OLTP
-- Use GROUPING SETS to select only the combinations you need
```

Mistake 3: Forgetting ORDER BY for readability

```
-- Results are unordered without explicit ORDER BY
-- Always sort by GROUPING() bits first to group subtotals appropriately
ORDER BY GROUPING(region), region, GROUPING(product), product;
```

Best Practices

1. Use ROLLUP for hierarchical data (year > month > day, country > region > city).
 2. Use CUBE for cross-tab analysis where all combinations are needed.
 3. Use GROUPING SETS for custom combinations — avoids generating unneeded rows.
 4. Always use GROUPING() to distinguish subtotal NULLs from data NULLs.
 5. Apply COALESCE with GROUPING() checks — not COALESCE alone.
 6. Limit CUBE to 3-4 dimensions in OLTP; use OLAP tools for more.
 7. Sort results with GROUPING() bits as the primary sort key.
-

Performance Considerations

```
-- ROLLUP and CUBE scan the table once and aggregate in memory
-- They are typically faster than equivalent UNION ALL queries

EXPLAIN ANALYZE
SELECT region, product, SUM(amount)
FROM sales
WHERE region IS NOT NULL
GROUP BY ROLLUP(region, product);
-- Look for: MixedAggregate or GroupAggregate with rollup flag

-- Indexes on group-by columns help:
CREATE INDEX idx_sales_region_product ON sales(region, product);
```

```
-- For very large tables, consider partial aggregation:  
-- Partition the data, ROLLUP each partition, combine with UNION ALL
```

Interview Questions & Answers

Q1: What is the difference between ROLLUP and CUBE?

A: ROLLUP generates subtotals along a defined hierarchy from left to right, producing N+1 groupings for N columns. CUBE generates all 2^N possible grouping combinations. ROLLUP is for hierarchical reporting (year > quarter > month); CUBE is for cross-dimensional analysis (region x product x period).

Q2: What is GROUPING SETS?

A: GROUPING SETS lets you specify exactly which grouping combinations you want in a single query, replacing multiple GROUP BY queries joined with UNION ALL. ROLLUP and CUBE are syntactic shortcuts that expand to specific GROUPING SETS patterns.

Q3: How do you tell the difference between a NULL from a rollup aggregate and a real NULL in the data?

A: Use the GROUPING() function. It returns 1 if the column is NULL due to rollup/cube aggregation, and 0 if the column is part of the current grouping (even if the actual value is NULL).

Q4: How many groupings does ROLLUP(A, B, C) produce?

A: ROLLUP with N columns produces N+1 groupings. ROLLUP(A, B, C) produces 4: (A,B,C), (A,B), (A), ().

Q5: How many groupings does CUBE(A, B, C) produce?

A: CUBE with N columns produces 2^N groupings. CUBE(A, B, C) produces $2^3 = 8$ groupings.

Q6: Can you combine ROLLUP and CUBE in the same query?

A: Yes, you can use GROUPING SETS with mixed patterns, or combine ROLLUP and CUBE using multiple GROUPING SETS. The Cartesian product of multiple GROUPING SETS clauses is computed.

Q7: What does GROUPING_ID() return?

A: GROUPING_ID() returns an integer where each bit corresponds to one GROUPING() call, packed from most significant to least significant bit. It provides a single integer to classify the aggregation level of each row.

Q8: How is ROLLUP better than UNION ALL for subtotals?

A: ROLLUP is more readable (one query vs many), guaranteed consistent (same filters and transformations), typically faster (single table scan), and maintained as one logical unit. UNION ALL forces you to repeat logic and keep multiple queries synchronized.

Hands-on Exercises

Exercise 1: Sales Hierarchy Report

Create a report showing revenue at three levels: region + product detail, region subtotals, and grand total. Label each row type.

```
-- Solution:  
SELECT
```

```

CASE WHEN GROUPING(region) = 1 THEN '*** GRAND TOTAL ***'
      ELSE COALESCE(region, 'Unknown')
END AS region,
CASE WHEN GROUPING(product) = 1 THEN '(Subtotal)'
      ELSE product
END AS product,
SUM(amount) AS revenue,
COUNT(*) AS transactions
FROM sales
WHERE region IS NOT NULL
GROUP BY ROLLUP(region, product)
ORDER BY GROUPING(region), region, GROUPING(product), product;

```

Exercise 2: Cross-Tab with CUBE

Generate a full cross-tab of revenue by region and product using CUBE, including all subtotals.

```

-- Solution:
SELECT
  COALESCE(CASE WHEN GROUPING(region) = 1 THEN 'ALL' ELSE region END, 'Unknown')
AS region,
  COALESCE(CASE WHEN GROUPING(product) = 1 THEN 'ALL' ELSE product END, 'Unknown')
AS product,
  SUM(amount) AS revenue,
  COUNT(*) AS count
FROM sales
WHERE region IS NOT NULL
GROUP BY CUBE(region, product)
ORDER BY GROUPING(region), region, GROUPING(product), product;

```

Exercise 3: Custom Grouping Sets

Write a query that produces: (rep, product) detail, (rep) subtotal, and grand total — but NOT a (product) subtotal.

```

-- Solution:
SELECT
  CASE WHEN GROUPING(rep_name) = 1 THEN 'GRAND TOTAL' ELSE rep_name END AS rep,
  CASE WHEN GROUPING(product) = 1 THEN '(Rep Total)' ELSE product END AS
product,
  SUM(amount) AS revenue
FROM sales
GROUP BY GROUPING SETS (
  (rep_name, product),
  (rep_name),
  ()
)
ORDER BY GROUPING(rep_name), rep_name, GROUPING(product), product;

```

Exercise 4: Time Hierarchy

Show monthly and yearly revenue totals for the sales data using ROLLUP.

```
-- Solution:
SELECT
  CASE WHEN GROUPING(yr) = 1 THEN 'GRAND TOTAL'
  ELSE yr::TEXT
  END AS year,
  CASE WHEN GROUPING(mo) = 1 THEN '(Year Total)'
  ELSE TO_CHAR(TO_DATE(mo::TEXT, 'MM'), 'Month')
  END AS month,
  SUM(amount) AS revenue,
  COUNT(*) AS sales
FROM (
  SELECT
    EXTRACT(YEAR FROM sale_date)::INT AS yr,
    EXTRACT(MONTH FROM sale_date)::INT AS mo,
    amount
  FROM sales
) AS ts
GROUP BY ROLLUP(yr, mo)
ORDER BY GROUPING(yr), yr, GROUPING(mo), mo;
```

Advanced Notes

Partial ROLLUP

You can partially apply ROLLUP to only some columns:

```
-- Roll up product within each rep, but keep rep fixed (no rep-level subtotal)
SELECT rep_name, product, SUM(amount)
FROM sales
WHERE region IS NOT NULL
GROUP BY rep_name, ROLLUP(product);
-- Produces: (rep, product) detail + (rep, NULL) per-rep subtotal
-- No grand total (rep is not inside ROLLUP)
```

GROUPING SETS Cross-Product

Multiple GROUPING SETS clauses in the same GROUP BY form a cross-product:

```
GROUP BY GROUPING SETS((A), (B)), GROUPING SETS((C), ())
-- Equivalent to:
GROUP BY GROUPING SETS((A,C), (A), (B,C), (B))
```

Cross-references

- [02_aggregations.md](#) — Foundation GROUP BY and aggregate functions
- [06_case_expressions.md](#) — CASE for labeling ROLLUP rows

- **01_window_functions_intro.md** (04_Advanced_SQL) — window functions for running totals without collapsing
- **06_advanced_aggregation.md** (04_Advanced_SQL) — statistical and hypothetical aggregates